Module 2: Lexing

Context-Free Languages

“How to do better than C++ without even trying”

0.

Assignment Review

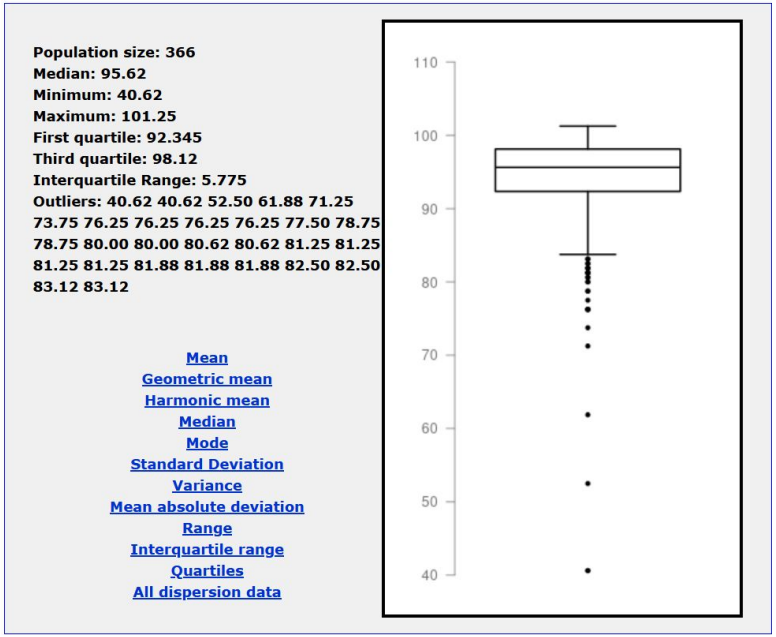
RegexTesting Final

RegexTesting Final

Average 93.7%, Median 95.6%

Overall matches expectations, outliers largely have partial implementations.

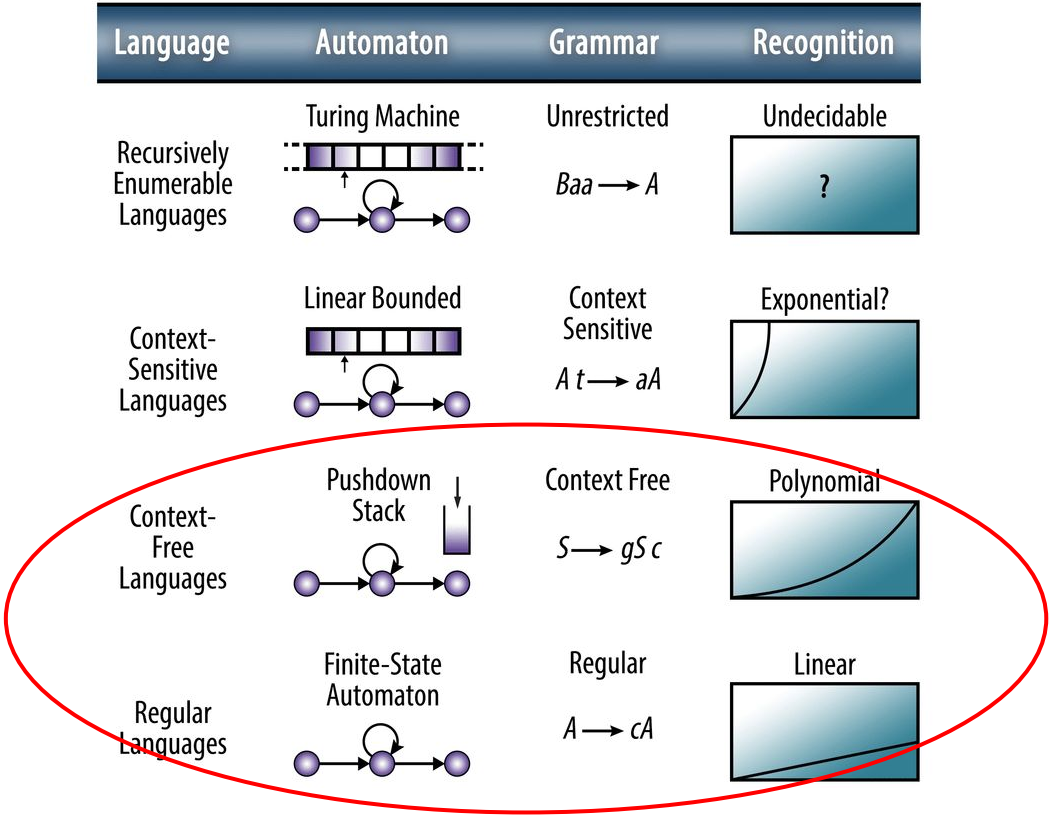
- Only 53.5% passed all Admin tests!
- Part 3: Core structure is a record, for listJavaFiles use (".*\.java", true).
- Notable: DATE Bonus, EMAIL Coverage, NUMBER Capitalized 'E'.
- Ask for help if needed; stay focused!



1.

Context-Free Grammars

(and their associated parse trees)



Chomsky Hierarchy

Context-Free Grammar

Definitions

A **context-free** grammar/language is:

- A grammar where all rules use a single, nonterminal character.
  - X ::= ε, X ::= aYa, Y ::= bXb
  - aX ::= cY (context-sensitive)
- A language that can be parsed using a push-down automaton.
- *Many* subclassifications exist based on optimal parsing algorithms.

Advantages/Tradeoffs

Primarily, this represents the category of "languages that are feasible to use".

- Easier for humans to read/understand.
  - Context-sensitive is English, so… yeah.
- Easier for machines to parse/validate
  - Syntax highlighting is a godsend!
- Many subclasses of context-free grammars exist specifically for optimal parsing.
- And we already know regular doesn't work.

```
1. S ::= aSa
2. S ::= bXb
3. X ::= cXc
4. X ::= ε //empty string "epsilon"

X → ε //empty string
X → cXc → cc
X → cXc → ccXcc → cccc
X ::= (cc)* //regex, neat!

S → bXb → b(cc)*b
S → aSa → abXba → ab(cc)*ba
S → aSa → aaSaa → aab(cc)*baa
S ::= aⁿb(cc)*baⁿ //equal #a's
```

```
1. S ::= A | B
2. A ::= AaE | EaE | EaA
3. B ::= BbE | EbE | EbB
4. E ::= aEbE | bEaE | ε

E → ε //empty string
E → aEbE → ab
E → aEbE → abbEaE → abba
E ::= #a's == #b's

A → EaE → a
A → AaE → EaEaE → aa
A ::= #a's > #b's
B ::= #a's < #b's

S ::= #a's != #b's
```

Context-Free Grammars

2.

Push-Down Automaton

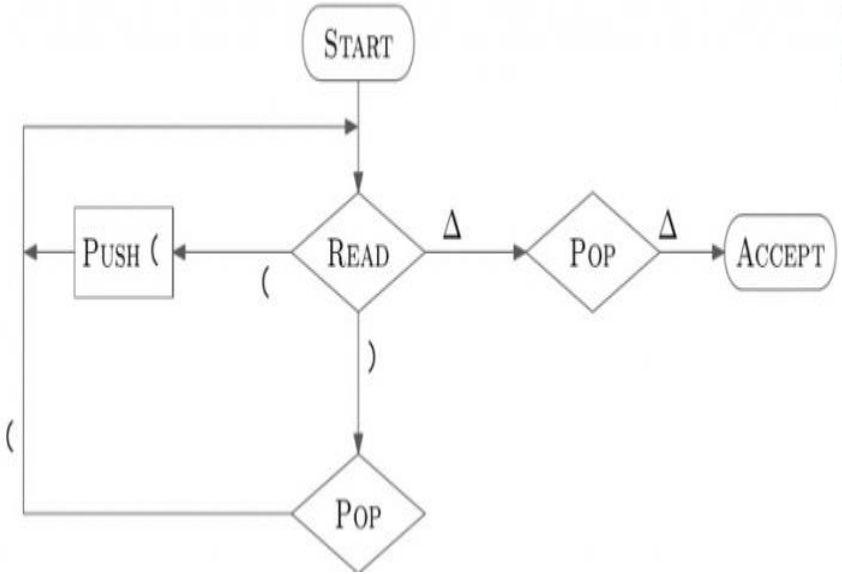
Imagine like, a FSM but with a Stack!

Push-Down Automaton

Push-Down Automaton

A **PDA** combines a **FSM** with a **stack** to store/recall info about previous input.

- The input, FSM state, and PDA stack determine valid transformations.
  - "(" -> push open
  - ")" + stack top == "(" -> pop
  - ")" + stack top != "("-> error
- Non-deterministic variants exist and are actually more powerful!



3.

Just-For-Fun Tangent

No quiz questions, I promise.

How can we prove grammar types?

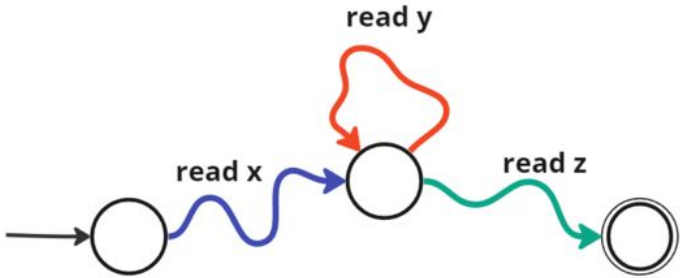
1. Prove <= X: Write a grammar, regex, FSM/PDA, etc.
2. Disprove < X: Contradiction via *pumping lemma*

Pumping Lemma

Definition (ish)

Informally, the *pumping lemma* states that sufficiently “long” strings must have an infinitely repeatable substring.

- The regex xy*z can “pump” y to generate xyz, xyyz, xyyyz, to infinity.
- Proof (ish): Given FSM and accepting string s using n states; if |s| > n there must a revisited state that can be “pumped” per pigeonhole principle.



Balanced Parens == Context-Free

Is <= Context-Free

Has been sufficiently proven via:

- The context-free grammar:
  - S ::= SS, S ::= (S), S ::= ε
  - (all rule LHS are nonterminals)
- The previous PDA, a machine that can recognize balanced parens.

Is > Regular

Proof by contradiction w/ pumping lemma:

- Let $s = (^p)^p$, representing p number of nested parens, which is balanced. Note |s| >= p.
- We divide s into substrings xyz such that |y| >= 1, |xy| <= p, and y can be pumped.
  - Since |xy| <= p, xy (and therefore y) contains only left parens.
  - Pumping y to xyyz must result in a string with left parens that are unbalanced.
  - Therefore, s is not pumpable/regular.

Module 2: Lexing

Context-Free Languages

“How to do better than C++ without even trying”

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals